



Map-in-Map Dynamic Routing

Lesson 09: Updating Packet Routing Tables Atomically Without Unloading Programs

Table of Contents

Lesson 09: Map-in-Map Dynamic Routing

1. Lesson Overview

2. Foundations & Core Technical Concepts

The Problem: Updating Running eBPF Configuration

What is Map-in-Map?

BPF_MAP_TYPE_ARRAY_OF_MAPS vs BPF_MAP_TYPE_HASH_OF_MAPS

How the Inner Map Reference Works

What Does the XDP Program See?

3. Problem Statement & Real-World Use Case

The Problem: Kubernetes Network Policy Rollout

Real Production Story: Cilium Network Policy Engine

4. Atomic Swap vs Sequential Update

5. Architecture Diagram

6. Routing Swap Sequence

7. Step-by-Step Code Walkthrough

The eBPF C Program (main.bpf.c)

The Go Loader (main.go)

8. Running the Lab

9. Verifier Errors & Common Mistakes

Error: inner map type mismatch

Common Mistake: Not holding a reference to the old inner map before swap

10. Performance Analysis

11. Best Practices

12. Summary

13. Further Reading

Technical Deep-Dive Q&A: Map-in-Map Dynamic Routing

1. Beginner Level

Q1. What is Map-in-Map? Why do we need it?

Q2. What is the difference between BPF_MAP_TYPE_ARRAY_OF_MAPS and BPF_MAP_TYPE_HASH_OF_MAPS?

2. Intermediate Level

Q3. How does the kernel validate inner maps before they are inserted into an outer map?

Q4. Walk through the lifetime of an inner map. What happens to it when it is swapped out?

3. Advanced Level

Q5. How does a BPF program safely fetch and query an inner map?

Hands-on Lab & Homework Assignments

1. Practical Exercises

Task 1: Trigger an Inner Map Mismatch

2. Coding Assignment: Dynamic Blacklist with Rollbacks

Requirements:

Lesson 09: Map-in-Map Dynamic Routing

1. Lesson Overview

In this lesson, you will build a **hot-reloadable packet routing system** using `BPF_MAP_TYPE_ARRAY_OF_MAPS` (Map-in-Map). You will learn how to atomically swap entire routing table configurations in a single system call — no XDP program restart, no dropped packets, no service interruption. This is how Cilium implements zero-downtime network policy upgrades in Kubernetes clusters with thousands of running pods.

2. Foundations & Core Technical Concepts

The Problem: Updating Running eBPF Configuration

Once an XDP program is loaded and attached to a network interface, it runs continuously. If your routing rules are stored in a regular BPF hash map, you can update individual entries. But what if you need to **atomically swap the entire routing table** — replacing 10,000 old rules with 10,000 new rules simultaneously?

With a standard BPF map:

- You must delete all old entries and insert all new entries one-by-one.
- During the migration window (which takes milliseconds), the XDP program reads **a mix of old and new rules** — a partially inconsistent state.
- For routing systems, this can cause packets to be forwarded to wrong backends during the swap.

What is Map-in-Map?

Map-in-Map allows you to store **BPF map file descriptors as values** inside another BPF map. The outer map holds references to inner maps. The XDP program looks up the outer map to get a reference to the inner map, then performs operations on the inner map.

This enables **atomic routing table swap**:

1. User-space builds a complete new inner map with all new rules.

2. User-space calls `bpf_map_update_elem` on the outer map to replace the inner map reference — **one atomic operation**.
3. From this moment, all XDP program lookups use the new inner map.
4. The old inner map is automatically freed when its last reference drops (when user-space closes its FD).

BPF_MAP_TYPE_ARRAY_OF_MAPS vs BPF_MAP_TYPE_HASH_OF_MAPS

Type	Outer Key	Use Case
<code>ARRAY_OF_MAPS</code>	Integer index (u32)	Fixed set of routing tables (e.g., by VLAN ID)
<code>HASH_OF_MAPS</code>	Arbitrary hash key	Per-tenant or per-namespace routing tables

How the Inner Map Reference Works

In C, when you declare a Map-in-Map, you specify a **template inner map** at compile time:

```
struct {
    __uint(type, BPF_MAP_TYPE_ARRAY_OF_MAPS);
    __uint(max_entries, 16);
    __type(key, __u32);
    __array(values, struct {
        __uint(type, BPF_MAP_TYPE_HASH);
        __uint(max_entries, 1024);
        __type(key, __u32);
        __type(value, __u32);
    });
} outer_map SEC(".maps");
```

The `__array(values, ...)` declaration tells the kernel the **exact type and size** of allowed inner maps. When user-space inserts a new inner map FD, the kernel validates it matches this template.

What Does the XDP Program See?

Inside the XDP program, `bpf_map_lookup_elem(&outer_map, &outer_key)` returns a **void *** pointer — a kernel-space reference to the inner map. You then pass this pointer directly to another `bpf_map_lookup_elem()` call:

```
void *inner_map = bpf_map_lookup_elem(&outer_map, &outer_key);
if (!inner_map) return XDP_PASS;
__u32 *route = bpf_map_lookup_elem(inner_map, &dest_ip);
```

3. Problem Statement & Real-World Use Case

The Problem: Kubernetes Network Policy Rollout

A Kubernetes cluster has 10,000 pods. Network policies specify which pods can communicate. When a new deployment rolls out and changes the network policy (e.g., "Service A v2 can now talk to Service B, but Service A v1 cannot"), Cilium must update routing tables across all 1,000 nodes simultaneously.

With standard map updates (one-by-one entry updates):

- The update propagates over ~500ms across the cluster.
- During propagation, some nodes enforce the new policy, others enforce the old.
- This inconsistency can cause intermittent connection failures during policy transitions.

With Map-in-Map atomic swap:

- The new inner map is fully built before any packet sees it.
- A single `bpf_map_update_elem` on the outer map atomically switches all XDP packet processing to the new policy.
- Transition time: **one syscall — microseconds**.

Real Production Story: Cilium Network Policy Engine

Cilium's eBPF dataplane uses **Hash-of-Maps** (`BPF_MAP_TYPE_HASH_OF_MAPS`) for per-endpoint network policies.

The outer map key is the **endpoint ID** (a Cilium-internal integer identifying a pod). The inner map contains the **allow/deny policy rules** for that endpoint. When a pod's network policy changes:

1. Cilium builds a completely new inner map with the updated rules.
2. Cilium inserts the new inner map into the outer map at the endpoint's key — atomic swap.
3. The old inner map is released.

From the BPF program's perspective, it simply queries `outer_map[endpoint_id][dest_ip]` — the routing logic never changes. Only the data changes, atomically.

Isovalent (the company behind Cilium) has stated this Map-in-Map pattern is critical for their Kubernetes CNI to achieve **sub-millisecond policy propagation** in clusters with thousands of nodes.

4. Atomic Swap vs Sequential Update

PROBLEM: Sequential entry update (INCONSISTENT during swap):

Time —————→

T0: Old map: {10.0.0.1→backend_A, 10.0.0.2→backend_A}

T1: Delete 10.0.0.1, Insert 10.0.0.1→backend_B ← PARTIAL STATE

T2: Delete 10.0.0.2, Insert 10.0.0.2→backend_B ← PARTIAL STATE

T3: New map: {10.0.0.1→backend_B, 10.0.0.2→backend_B}

Packets arriving at T1 or T2 see inconsistent routing!

SOLUTION: Map-in-Map atomic swap (ALWAYS CONSISTENT):

Time —————→

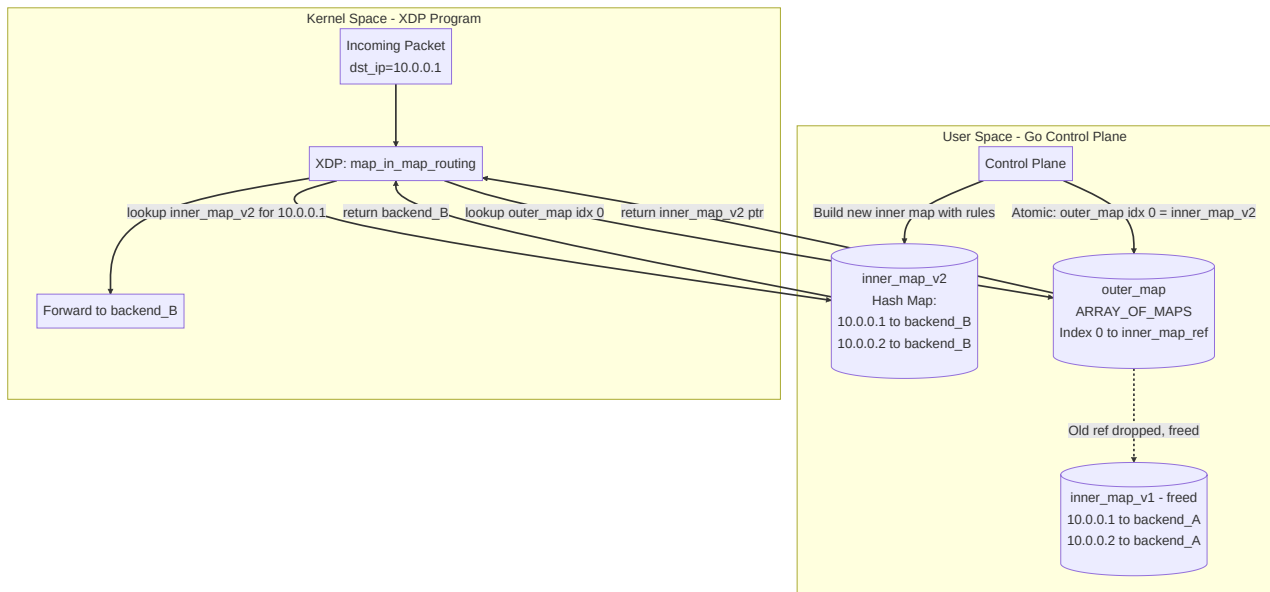
T0: `outer_map[0] → inner_map_A {10.0.0.1→A, 10.0.0.2→A}`

Build: `inner_map_B = {10.0.0.1→B, 10.0.0.2→B}` (in background)

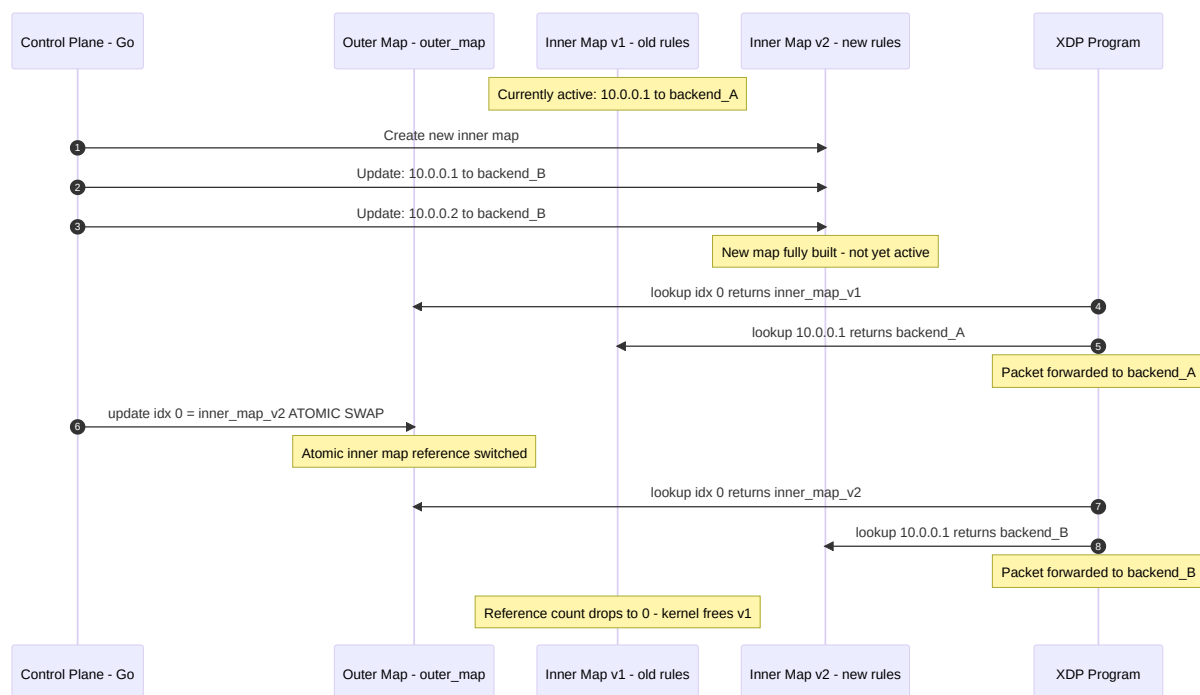
T3: `outer_map[0] = inner_map_B` ← ONE ATOMIC SWAP

All packets either see all-A or all-B. Never a partial state.

5. Architecture Diagram



6. Routing Swap Sequence



7. Step-by-Step Code Walkthrough

The eBPF C Program (`main.bpf.c`)

```
#include <vmlinux.h>
#include <bpf/bpf_helpers.h>
#include <bpf/bpf_endian.h>

// Outer map: Array-of-Maps
// Each slot holds a reference to one inner routing table
// Template inner map must be declared here so the kernel knows the allowed inner ty
struct {
    __uint(type, BPF_MAP_TYPE_ARRAY_OF_MAPS);
    __uint(max_entries, 16);    // 16 possible routing tables (e.g., by VLAN)
    __type(key, __u32);        // Outer key: routing table index
    // Template for allowed inner maps:
    __array(values, struct {
        __uint(type, BPF_MAP_TYPE_HASH);
        __uint(max_entries, 10240); // Max 10K routes per table
        __type(key, __u32);          // Key: destination IP (network byte order)
        __type(value, __u32);        // Value: next-hop IP or backend index
    });
} outer_map SEC(".maps");

SEC("xdp")
int map_in_map_routing(struct xdp_md *ctx) {
    void *data_end = (void *)(long)ctx->data_end;
    void *data = (void *)(long)ctx->data;

    struct ethhdr *eth = data;
    if ((void *)(eth + 1) > data_end) return XDP_PASS;
    if (eth->h_proto != bpf_htons(ETH_P_IP)) return XDP_PASS;

    struct iphdr *iph = (void *)(eth + 1);
    if ((void *)(iph + 1) > data_end) return XDP_PASS;

    // Select routing table 0 (VLAN 0 / default route table)
    __u32 outer_key = 0;

    // Look up the current active inner map in slot 0 of the outer map
    // Returns a void * pointer to the inner map (a kernel BPF map object)
    void *inner_map = bpf_map_lookup_elem(&outer_map, &outer_key);
    if (!inner_map) return XDP_PASS; // No routing table loaded yet
```

```
// Use the destination IP as the inner map key
__u32 dst_ip = iph->daddr;    // Already in network byte order

// Perform a lookup in the inner routing table
__u32 *next_hop = bpf_map_lookup_elem(inner_map, &dst_ip);
if (next_hop) {
    bpf_printk("Route match: dst=%x → next_hop=%x", dst_ip, *next_hop);
    // In a real implementation, we'd modify the packet's dst MAC and redirect
    // For this lesson, we just log the routing decision
} else {
    bpf_printk("No route for dst=%x - default pass", dst_ip);
}

return XDP_PASS;
}

char LICENSE[] SEC("license") = "GPL";
```

The Go Loader (`main.go`)

```
package main

import (
    "encoding/binary"
    "fmt"
    "log"
    "net"
    "os"
    "os/signal"
    "syscall"
    "time"

    "github.com/cilium/ebpf"
    "github.com/cilium/ebpf/link"
)

func main() {
    spec, err := ebpf.LoadCollectionSpec("main.bpf.o")
    if err != nil {
        log.Fatalf("load spec: %v", err)
    }

    var objs struct {
        Prog      *ebpf.Program `ebpf:"map_in_map_routing"`
        OuterMap *ebpf.Map     `ebpf:"outer_map"`
    }

    if err := spec.LoadAndAssign(&objs, nil); err != nil {
        log.Fatalf("load objects: %v", err)
    }

    defer objs.Prog.Close()
    defer objs.OuterMap.Close()

    l, err := link.AttachXDP(link.XDPOpts{
        Interface: 1,
        Program:   objs.Prog,
    })
    if err != nil {
        log.Fatalf("attach XDP: %v", err)
    }
    defer l.Close()

    // Build and activate the FIRST routing table (v1)
    log.Println("Loading routing table v1... ")
    if err := loadRoutingTable(objs.OuterMap, 0, map[string]string{
```

```

        "8.8.8.8": "10.0.0.1",    // Google DNS → backend 10.0.0.1
        "1.1.1.1": "10.0.0.2",    // Cloudflare DNS → backend 10.0.0.2
    }); err != nil {
        log.Fatalf("load v1: %v", err)
    }
    fmt.Println("Routing table v1 active. 8.8.8.8→10.0.0.1, 1.1.1.1→10.0.0.2")

    // After 5 seconds, atomically swap to routing table v2
    time.AfterFunc(5*time.Second, func() {
        log.Println("Atomically swapping to routing table v2 ...")
        if err := loadRoutingTable(objs.OuterMap, 0, map[string]string{
            "8.8.8.8": "10.0.0.5",    // Google DNS now → backend 10.0.0.5 (new)
            "1.1.1.1": "10.0.0.6",    // Cloudflare DNS now → backend 10.0.0.6 (new)
        }); err != nil {
            log.Printf("swap failed: %v", err)
            return
        }
        fmt.Println("Routing table v2 now active. 8.8.8.8→10.0.0.5, 1.1.1.1→10.0.0.6")
        fmt.Println("Old table v1 freed automatically.")
    })

    sig := make(chan os.Signal, 1)
    signal.Notify(sig, os.Interrupt, syscall.SIGTERM)
    <-sig
}

// loadRoutingTable creates a new inner map, populates it with routes,
// then atomically replaces the inner map at the given outer slot.
func loadRoutingTable(outerMap *ebpf.Map, slot uint32, routes map[string]string) error {
    // Create a new inner hash map matching the template declared in the C program
    innerSpec := &ebpf.MapSpec{
        Type:      ebpf.Hash,
        KeySize:    4,    // uint32 (IP address)
        ValueSize:  4,    // uint32 (next-hop IP)
        MaxEntries: 10240,
    }
    innerMap, err := ebpf.NewMap(innerSpec)
    if err != nil {
        return fmt.Errorf("create inner map: %w", err)
    }
    defer innerMap.Close() // Close our reference; outer map holds another reference

    // Populate the inner map with routing rules
    for dst, nexthop := range routes {
        dstIP := binary.BigEndian.Uint32(net.ParseIP(dst).To4())
        nextIP := binary.BigEndian.Uint32(net.ParseIP(nexthop).To4())
        if err := innerMap.Update(&dstIP, &nextIP, ebpf.UpdateAny); err != nil {

```

```
        return fmt.Errorf("insert route %s→%s: %w", dst, nexthop, err)
    }
}

// ATOMIC SWAP: replace the inner map reference at slot `slot` in the outer map
// This single syscall atomically switches which inner map the XDP program uses
if err := outerMap.Update(&slot, innerMap, ebpf.UpdateAny); err != nil {
    return fmt.Errorf("swap inner map: %w", err)
}
return nil
}
```

8. Running the Lab

```
cd series/series-09/source-code
make
sudo ./loader
```

Expected output:

```
Loading routing table v1...
Routing table v1 active. 8.8.8.8→10.0.0.1, 1.1.1.1→10.0.0.2
[5 seconds later]
Atomically swapping to routing table v2...
Routing table v2 now active. 8.8.8.8→10.0.0.5, 1.1.1.1→10.0.0.6
Old table v1 freed automatically.
```

Generate traffic in another terminal to see routing decisions:

```
# Traffic to 8.8.8.8 will trigger the route lookup
ping -c 10 8.8.8.8

# Read routing decisions from kernel log
sudo cat /sys/kernel/debug/tracing/trace_pipe
```

9. Verifier Errors & Common Mistakes

Error: inner map type mismatch

```
Error: EINVAL: inner_map type must match template
```

Cause: The inner map you're inserting into the outer map has different `type`, `key_size`, `value_size`, or `max_entries` than the template declared in `__array(values, ...)`. **Fix:** Create the inner map from the outer map's inner spec: `outerMap.InnerMapSpec()`.

Common Mistake: Not holding a reference to the old inner map before swap

If you swap the outer map reference without keeping a user-space FD to the old inner map, the old map is freed immediately. If you wanted to compare old vs new state, obtain the old map first:

```
var oldMapFD uint32
outerMap.Lookup(&slot, &oldMapFD) // Get old map's FD before swapping
// ... do the swap ...
// Now you can still examine oldMapFD until you close it
```

10. Performance Analysis

Metric	Sequential Entry Update	Map-in-Map Atomic Swap
Swap duration	$O(N)$ — one syscall per entry	$O(1)$ — one syscall
Inconsistency window	Milliseconds (for 10K rules)	Zero (atomic)
Packet drops during swap	Possible (wrong route)	None

11. Best Practices

- **Pre-build inner maps in user-space before swapping:** Populate all entries in the new inner map before inserting it into the outer map. This ensures the XDP program never sees a partially filled routing table.
 - **Keep N+1 inner maps:** Maintain the current active map plus one pre-built "staging" map ready to swap. This enables instant rollback by swapping back to the previous inner map.
 - **Use HASH_OF_MAPS for per-tenant routing:** Multi-tenant systems should use a hash outer map keyed by tenant ID, with one inner routing table per tenant.
-

12. Summary

- Map-in-Map stores BPF map file descriptors as values in an outer map.
 - The XDP program retrieves the inner map pointer via `bpf_map_lookup_elem` on the outer map.
 - User-space can atomically swap the inner map reference with a single `bpf_map_update_elem` call.
 - This enables zero-downtime routing table updates with no packet inconsistency window.
 - Cilium uses this for atomic Kubernetes network policy propagation.
-

13. Further Reading

- **Cilium Policy Maps:** [Cilium eBPF Datapath Architecture](#)
- **Linux Map-in-Map Kernel Docs:** [BPF Map-in-Map Documentation](#)
- **Isovalent eBPF Labs:** [Isovalent eBPF Playground](#)

Technical Deep-Dive Q&A: Map-in-Map Dynamic Routing

1. Beginner Level

Q1. What is Map-in-Map? Why do we need it?

Answer: Map-in-Map is an eBPF feature that allows storing **file descriptors of BPF maps as values** in another "outer" BPF map. This is critical for scenarios where we need to update entire routing tables, firewall policy sets, or namespaces dynamically. Instead of updating entries sequentially (which creates a race condition where the program sees a mix of old and new rules), we can populate a completely new "inner" map and swap its reference in the "outer" map atomically with a single system call.

Q2. What is the difference between `BPF_MAP_TYPE_ARRAY_OF_MAPS` and `BPF_MAP_TYPE_HASH_OF_MAPS`?

Answer:

- **ARRAY_OF_MAPS** : The outer map is an array. The outer key is a simple index (0 to MaxEntries-1). This is suitable for scenarios where you have a fixed set of routing tables (e.g., matching a VLAN ID or CPU core index).
 - **HASH_OF_MAPS** : The outer map is a hash table. The outer key can be arbitrary data (e.g., a source IP, tenant ID, or namespace ID). This is suitable for multi-tenant, dynamically scaled virtual routing tables.
-

2. Intermediate Level

Q3. How does the kernel validate inner maps before they are inserted into an outer map?

Answer: When defining an outer Map-in-Map, you must specify a **template inner map** definition at compilation time. The kernel validates that any inner map loaded at runtime matches the template's configuration:

- `map_type` (e.g., HASH, ARRAY)
- `key_size`
- `value_size`
- `max_entries`

If you try to insert an inner map that doesn't match this template, `bpf_map_update_elem` returns `-EINVAL`.

Q4. Walk through the lifetime of an inner map. What happens to it when it is swapped out?

Answer:

1. User-space creates the inner map and populates its rules.
2. User-space updates the outer map's slot to point to the new inner map. This increases the inner map's reference count.
3. User-space closes its local file descriptor for the inner map. The outer map still holds a reference, keeping the map alive.
4. When user-space swaps it out later, the reference count drops. Once any in-flight eBPF programs finish executing and release their pointers, the kernel automatically deallocates and frees the old inner map.

3. Advanced Level

Q5. How does a BPF program safely fetch and query an inner map?

Answer:

```
void *inner = bpf_map_lookup_elem(&outer_map, &outer_key);  
if (!inner) return XDP_PASS; // Defensive safety check  
  
__u32 *route = bpf_map_lookup_elem(inner, &inner_key);
```

The first lookup returns a `void *` pointer referencing the inner map structure. eBPF helpers accept this pointer as the map target for secondary lookups.

Hands-on Lab & Homework Assignments

1. Practical Exercises

Task 1: Trigger an Inner Map Mismatch

Try creating an inner map with `MaxEntries: 100` instead of the 10240 template size. What error does `loadRoutingTable` return?

2. Coding Assignment: Dynamic Blacklist with Rollbacks

Objective: Build a firewall policy table using `HASH_OF_MAPS` where the outer map key is the Tenant ID, and the inner map stores blacklisted IPs.

Requirements:

1. Implement a Go CLI supporting:
 - `blacklist <tenant> <ip>`
 - `swap <tenant>` — swap in the new policy
 - `rollback <tenant>` — restore the previous inner map descriptor
2. Verify traffic from blacklisted IPs gets dropped instantly after swapping.